

CS111

Introduction to Computing Science

Functions

- Recap

```
double cube_volume(double side_length)
{
    double volume = side_length * side_length * side_length;
    return volume;
}
```

- All function have a return **data type**.
- It is the **data type** of the **return value**

Functions

- Recap

```
double cube_volume(double side_length)
{
    double volume = side_length * side_length * side_length;
    return volume;
}
```

- Function have a **function name**.
- Follows the same rules we applied to our variables.
- Use a name that tell what the function is expected to do.

Functions

- Recap

```
double cube_volume(double side_length)
{
    double volume = side_length * side_length * side_length;
    return volume;
}
```

- Function may have **arguments** or **parameter**
- You need to define their **type**.
- They are used like variables inside a function
- Their values are passed from the calling function

Functions

- Recap

```
double cube_volume(double side_length)
{
    double volume = side_length * side_length * side_length;
    return volume;
}
```

- The **function body** follows between curly brackets
- It defines what a function does
- It behaves like a program on its own

Functions

- Recap

```
double cube_volume(double side_length)
{
    double volume = side_length * side_length * side_length;
    return volume;
}
```

- The **function body** follows between curly brackets
- It defines what a function does
- It behaves like a program on its own

Functions

- Recap

```
double cube_volume(double side_length)
{
    double volume = side_length * side_length * side_length;
    return volume;
}
```

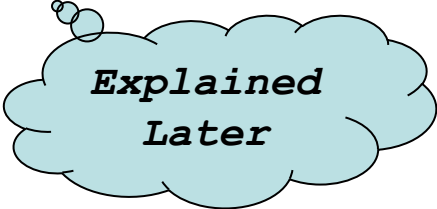
- The function body follows between curly brackets
- It defines what a function does
- It behaves like a program on its own
- Variables are either **declared inside** or **parameters**

Functions

■ Recap

```
double cube_volume(double side_length)
{
    double volume = side_length * side_length * side_length;
    return volume;
}
```

- A function may have a **return value**
- It may return a variable, parameter, constant or result of an expression.
- You have to return a value, unless the return type is *void*.
- Many compilers will **not warn** if you forget it.



***Explained
Later***

Functions

- Recap

```
double length=4;
double result1,result2,result3;
result1 = cube_volume(length) ;
result2 = cube_volume(5) ;
result3 = cube_volume(length + 5) ;
```

- When calling a function **variables**, **literals**, or **expressions** may be passed.
- The types should match the type of the parameter
- If you assign the return value, its type should match.

Functions Without Return Values

Consider the task of writing a string with the following format around it.

Any string could be used.

For example, the string "**He11o**" would produce:

```
-----  
!He11o!  
-----
```

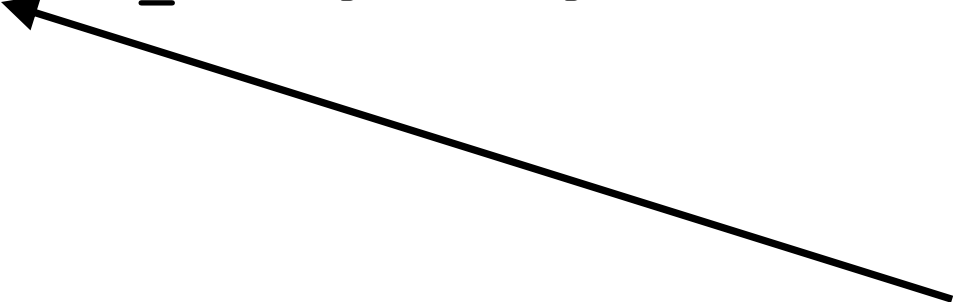
Functions Without Return Values

The void Type

A function for this task can be defined as follows:

```
void box_string(string str)
```

Notice the return type of this function:



void

Functions Without Return Values

The `void` Type

This kind of function is called a *`void` function*.

```
void box_string(string str)
```

Use a return type of **`void`** to indicate that a function does not return a value.

`void` functions are used to

simply do a sequence of instructions

- They do not return a value to the caller.

Functions Without Return Values

The `void` Type

void functions are used ***only*** to do a sequence of instructions.

Functions Without Return Values

The `void` Type

- Print a line that contains the '-' character $n + 2$ times, where n is the length of the string.
- Print a line containing the string, surrounded with a ! to the left and right.
- Print another line containing the - character $n + 2$ times.

```
-----  
!Hello!  
-----
```

Functions Without Return Values

The void Type

```
void box_string(string str)
{
    int n = str.length();
    for (int i = 0; i < n + 2; i++){ cout << "-"; }
    cout << endl;
    cout << "!" << str << "!" << endl;
    for (int i = 0; i < n + 2; i++) { cout << "-"; }
    cout << endl;
}
```

Note that this function doesn't compute any value.

It performs some actions and then returns to the caller
– without returning a value.
(The return occurs at the end of the block.)

Functions Without Return Values

The void Type

Because there is no return value, you cannot use `box_string` in an expression.

You can make this call kind of call:

```
box_string("Hello");
```

but not this kind:

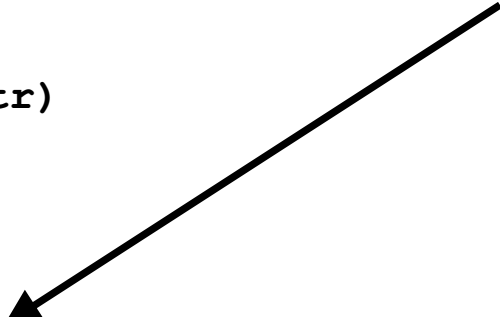
```
result = box_string("Hello");  
// Error: box_string doesn't  
//          return a result.
```


Functions Without Return Values

The void Type

If you want to return from a **void** function before reaching the end, you use a **return** statement without a value. For example:

```
void box_string(string str)
{
    int n = str.length();
    if (n == 0)
    {
        return;    // Return immediately
    }
    . . . // None of these statements
           // will be executed
}
```



Designing Functions

Turn Repeated Code into Functions

When you write nearly identical code multiple times,
you should probably introduce a function.

Designing Functions

Turn Repeated Code into Functions

Consider how similar the following statements are:

```
do{  
    cout << "Enter the mark for exam1:  ";  
    cin >> exam1;  
} while (exam1 < 0 || exam1 > 100);
```

```
do{  
    cout << "Enter the mark for exam2:  ";  
    cin >> exam2;  
} while (exam2 < 0 || exam2 > 100);
```

```
do{  
    cout << "Enter the mark for exam3:  ";  
    cin >> exam3;  
} while (exam3 < 0 || exam3 > 100);
```

Designing Functions

Turn Repeated Code into Functions

Move the *common behavior* into *one* function

```
double read_valid_mark(string exam_name){  
  
    double exam;  
    do  
    {  
        cout << "Enter the mark for " << exam_name << ":";  
        cin >> exam;  
    }  
    while (exam < 0 || exam > 100);  
  
    return exam;  
  
}
```

Designing Functions

Turn Repeated Code into Functions

Then we can use this function as many times as we need:

```
exam1 = read_valid_mark("Exam 1");  
exam2 = read_valid_mark("Exam 2");  
exam3 = read_valid_mark("Exam 3");
```

Note how the code has become much easier to understand.

And we are not rewriting code

– code reuse!

Designing Functions

Turn Repeated Code into Functions

Perhaps we can make this function even better:

```
exam1 = read_valid_mark("Exam 1");
```

Can we use this function to get a valid mark between *low* and *high*.

We can modify the code to take two additional parameters:

```
double read_valid_mark(string exam_name, int low, int high)
```

Designing Functions

Turn Repeated Code into Functions

Improve one function rather than multiple occurrences

```
double read_valid_mark(string exam_name, int low, int high){  
  
    double exam;  
    do  
    {  
        cout << "Enter the mark for " << exam_name << ":";  
        cin >> exam;  
    }  
    while (exam < low || exam > high);  
  
    return exam;  
  
}
```

Designing Functions

Turn Repeated Code into Functions

We can use this function as many times as we need, passing in the end points of the valid range:

```
exam1 = read_valid_mark("Exam 1", 0, 30) ;  
exam2 = read_valid_mark("Exam 2", 0, 30) ;  
exam3 = read_valid_mark("Exam 3", 0, 40) ;
```

Note how the code has become even better.

And we are still not rewriting code

– code reuse!

Tracing Functions

When you design a complex set of functions, it is a good idea to carry out a manual walkthrough before entrusting your program to the computer.

This process is called *tracing* your code.

Tracing Functions

```
0 int doublenum(int num){
1     num = 2 * num;
2     return num;
3 }
4
5 int main(){
6     int number = 21;
7     number = doublenum(number);
8     cout << "Result: " << number;
9     return 0;
}
```

	line	number	Comment
main			

Tracing Functions

```
0 int doublenum(int num){  
1     num = 2 * num;  
2     return num;  
3 }  
4  
5 int main(){  
6     int number = 21;  
7     number = doublenum(number);  
8     cout << "Result: " << number;  
9     return 0;  
}
```

	line	number	Comment
main	6	21	declare number

Tracing Functions

```
0 int doublenum(int num){
1     num = 2 * num;
2     return num;
3 }
4
5 int main(){
6     int number = 21;
7     number = doublenum(number);
8     cout << "Result: " << number;
9     return 0;
}
```

	line	number	Comment
main	6	21	declare number
	7	"	call doublenum

Tracing Functions

```

0 int doublenum(int num){
1     num = 2 * num;
2     return num;
3 }
4
5 int main(){
6     int number = 21;
7     number = doublenum(number);
8     cout << "Result: " << number;
9     return 0;
}
```

	line	number	Comment		line	num	Comment
main	6	21	declare number	doublenum			
	7	"	call doublenum				

Tracing Functions

```

0  int doublenum(int num){
1      num = 2 * num;
2      return num;
3  }
4
5  int main(){
6      int number = 21;
7      number = doublenum(number);
8      cout << "Result: " << number;
9      return 0;
10 }

```

	line	number	Comment		line	num	Comment
main	6	21	declare number				
	7	"	call doublenum				
				doublenum	0	21	use value of number

Tracing Functions

```

0  int doublenum(int num){
1      num = 2 * num;
2      return num;
3  }
4
5  int main(){
6      int number = 21;
7      number = doublenum(number);
8      cout << "Result: " << number;
9      return 0;
10 }

```

	line	number	Comment		line	num	Comment
main	6	21	declare number		0	21	use value of number
	7	"	call doublenum	→	1	42	2 * 21 =42

Tracing Functions

```

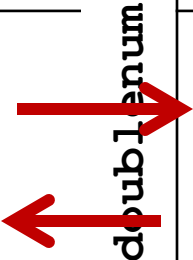
0 int doublenum(int num){
1   num = 2 * num;
2   return num;
3 }
4
5 int main(){
6   int number = 21;
7   number = doublenum(number);
8   cout << "Result: " << number;
9   return 0;
}
```

	line	number	Comment		line	num	Comment
main	6	21	declare number	doublenum 	0	21	use value of number
	7	"	call doublenum		1	42	2 * 21 =42
					2	"	return value 42

Tracing Functions

```
0 int doublenum(int num){
1     num = 2 * num;
2     return num;
3 }
4
5 int main(){
6     int number = 21;
7     number = doublenum(number);
8     cout << "Result: " << number;
9     return 0;
}
```

	line	number	Comment		line	num	Comment
main	6	21	declare number		0	21	use value of number
	7	"	call doublenum		1	42	2 * 21 =42
					2	"	return value 42



Tracing Functions

```

0 int doublenum(int num){
1     num = 2 * num;
2     return num;
3 }
4
5 int main(){
6     int number = 21;
7     number = doublenum(number);
8     cout << "Result: " << number;
9     return 0;
}
```

	line	number	Comment		line	num	Comment
main	6	21	declare number		0	21	use value of number
	7	"	call doublenum	→	1	42	2 * 21 = 42
	7	42	assign value 42	←	2	"	return value 42

Tracing Functions

```

0 int doublenum(int num){
1     num = 2 * num;
2     return num;
3 }
4
5 int main(){
6     int number = 21;
7     number = doublenum(number);
8     cout << "Result: " << number;
9     return 0;
}
```

	line	number	Comment		line	num	Comment
main	6	21	declare number		0	21	use value of number
	7	"	call doublenum	→	1	42	2 * 21 = 42
	7	42	assign value 42	←	2	"	return value 42
	8	"	print Result: 42				

Tracing Functions

```

0  int doublenum(int num){
1      num = 2 * num;
2      return num;
3  }
4
5  int main(){
6      int number = 21;
7      number = doublenum(number);
8      cout << "Result: " << number;
9      return 0;
   }

```

	line	number	Comment		line	num	Comment
main	6	21	declare number		0	21	use value of number
	7	"	call doublenum	→	1	42	2 * 21 = 42
	7	42	assign value 42	←	2	"	return value 42
	8	"	print Result: 42				
	9	"	the end				

Tracing Functions

When you design a complex set of functions, it is a good idea to carry out a manual walkthrough before entrusting your program to the computer.

This process is called *tracing* your code.

You can also trace each of your functions separately.

Tracing Functions

```
0 int doublenum(int num){  
1   num = 2 * num;  
2   return num;  
3 }
```

doublenum	line	num	Comment
	0	42	assume num is 42
	1	84	2 * 42 = 84
	2	"	return value 82

You can assume any input that matches the type.

Where to put your functions?

Option 1

- Place your function **before** the main function
- Requires to put the function **before** it first use by any other function.

Where to put your functions?

```
int doublenum(int num){  
    return 2 * num;  
}  
  
int quadruplenum(int num){  
    return 2 * doublenum(num);  
}  
  
int main(){  
    int number = 21;  
    number = quadruplenum(number);  
    cout << "Result: " << number;  
    return 0;  
}
```

Note: Function can call other functions, can call other functions.

Where to put your functions?

Option 1

- Place your function **before** the main function
- Requires to put the function **before** it first use by any other function.
- Cumbersome when you have many functions.
- Impossible if there is a circular dependency.

function `chicken` calls function `egg`, and function `egg` calls function `chicken`. Which comes first? `Chicken` or `egg`?

More about these “recursive” functions next week.

Where to put your functions?

Option 2

- Place your function **after** the main function
- **Problem:** The compiler works top down. It will complain about undefined functions..
- Unless ... You put a so-called *prototype* **before** all functions.

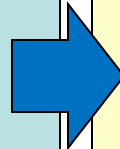
A prototype tells the compiler that it should expect a function with a particular return type, name, and list of parameters, **later**.

A *prototype* is a function without its body.

Where to put your functions

Functions

```
int doublenum(int num){  
    return 2 * num;  
}  
  
int quadruplenum(int num){  
    return 2 * doublenum(num);  
}
```



Prototype

```
int doublenum(int num);  
  
int quadruplenum(int num);
```

Do not forget the semicolon at the end!

Where to put your functions?

```
int quadruplenum(int num);  
int doublenum(int num);  
  
int main(){  
    int number = 21;  
    number = quadruplenum(number);  
    cout << "Result: " << number;  
    return 0;  
}  
int doublenum(int num){  
    return 2 * num;  
}  
int quadruplenum(int num){  
    return 2 * doublenum(num);  
}
```

- Prototypes come first, in any order.
- Functions come next, in any order.

Stepwise Refinement

- One of the most powerful strategies for problem solving is the process of *stepwise refinement*.
- To solve a difficult task, break it down into simpler tasks.
- Then keep breaking down the simpler tasks into even simpler ones, until you are left with tasks that you know how to solve.
- Do your homework. Think about pseudo code.

Good Design – Keep Functions Short

- There is a certain cost for writing a function:
 - You need to design, code, and test the function.
 - The function needs to be documented.
 - You need to spend some effort to make the function *reusable* rather than tied to a specific context.

Good Design – Keep Functions Short

- And you should keep your functions short.
- As a rule of thumb, a function that is so long that its will not fit on a single screen in your development environment should probably be broken up.
- Break the code into other functions

Quiz

7. Which of the following functions has *return type int*?

(A)

```
int foo(double number){  
  
    return number/2;  
}
```

(B)

```
float foo(int number){  
  
    return number/2;  
}
```

(C)

```
void foo(int number){  
    int result = number/2;  
    return result;  
}
```

(D)

```
double foo(int number){  
    int result = number/2;  
    return int;  
}
```

- (A) Function (A).
- (B) Function (B).
- (C) Function (C).
- (D) Function (D).
- (E) None of the above.

Quiz

9. Which of the following statements about *functions* is correct.
- (A) A function is a sequence of instructions with a name.
 - (B) Every statement in C++ is a function.
 - (C) All functions take integer input, and return integer output.
 - (D) The `main` function in C++ is not a function.
 - (E) All of the above.

Quiz

10. What are common reasons to use functions?

- (A) Functions are used to break up a program into smaller parts.
- (B) Functions are used to enable reuse of common and useful sets of instructions.
- (C) Functions are used to make programs more readable and maintainable.
- (D) Functions are used to make it possible to test and develop parts in isolation.
- (E) All of the above.

11. Which of the following function definitions is syntactically correct, i.e. which compiles without error?

(A) <pre>int mymax(int num1;int num2;){ if(num1 >num2){ return = num1; } return = num2; }</pre>	(B) <pre>void mymax(int num1,int num2){ if(num1 > num2){ returns(num1); } returns(num2); }</pre>
(C) <pre>int mymax(int num1,int num2){ if(num1 >num2){ return num1; } return num2; }</pre>	(D) <pre>long mymax(int num1;int num2;){ if(num1 >num2){ returns num1; } returns num2; }</pre>

- (A) Function (A).
- (B) Function (B).
- (C) Function (C).
- (D) Function (D).
- (E) None of the above.